

Java Collections Framework

Java Collections Framework

Oracle's Java documentation describes its collections framework as:

"A unified architecture for representing and manipulating collections, enabling collections to be manipulated independently of implementation details."

The **Java Collections Framework (JCF)** is a **set of interfaces, classes, and algorithms** that provide **standard ways to store, access, and manipulate groups of objects**.

All collection frameworks in java include the following three elements:

1. Interfaces
2. Implementation or classes
3. Collection Algorithms

The framework is primarily contained in the **java.util** package.

Need of Collections Framework

Without collections, you would rely on arrays.

But arrays have limitations:

- Fixed size (you can't resize easily).
- No built-in methods for searching, sorting, or manipulating.
- Only sequential access.

Collections solve these problems by giving dynamic, flexible, reusable data structures.

Key Benefits of Java Collections Frameworks

Provides **standardized data structures** (ArrayList, HashSet, HashMap, etc.).

Reduces **manual coding** for storing, searching, sorting, or manipulating data.

Provides **algorithms** for operations like sorting and searching.

Supports **type safety** using generics.

Components of Java Collections Framework

Interfaces - Define collection types

- List
- Set
- Queue
- Map

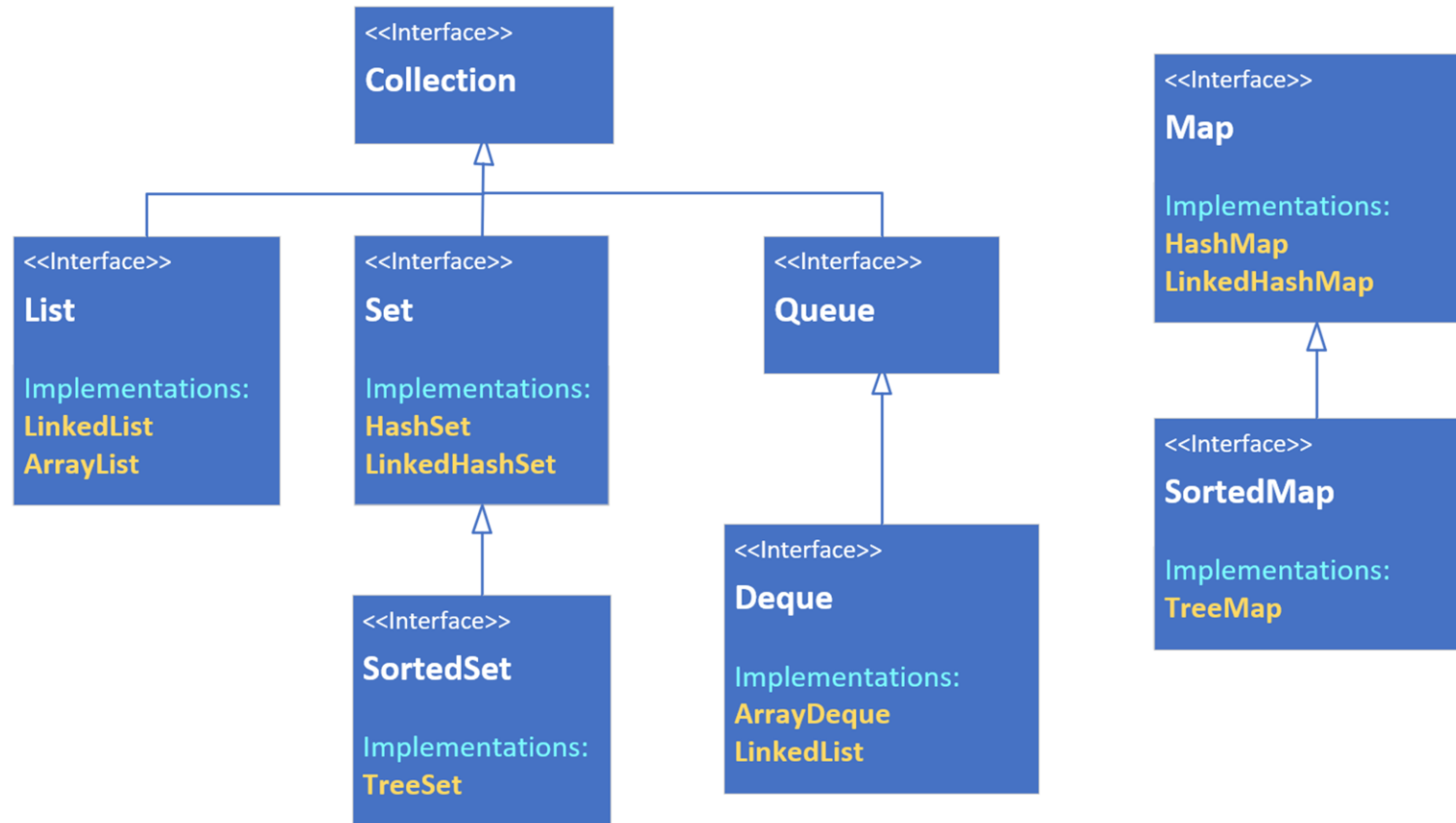
Classes - Actual implementations you use in code

- For Lists : ArrayList , LinkedList
- For Sets : HashSet, LinkedHashSet, TreeSet
- For Maps : HashMap, TreeMap

Algorithms - Built-in methods in the Collections class

- sort()
- binarySearch()
- reverse(), shuffle()
- fill(), copy()
- min(), max()

Hierarchy of Java Collections Framework



Core Interfaces in the Framework

Interface	Description	Implementations
Collection	Root interface for all collections	ArrayList, LinkedList, HashSet, TreeSet
List	Ordered collection, allows duplicates, indexed	ArrayList, LinkedList
Set	Collection of unique elements	HashSet, LinkedHashSet, TreeSet
Queue	FIFO order (first-in-first-out)	LinkedList, PriorityQueue
Deque	Double-ended queue	ArrayDeque, LinkedList
Map	Key-value pairs (not a Collection)	HashMap, LinkedHashMap, TreeMap

Note: **Map** is part of JCF but **does not extend Collection**.

Collection Interface

It is the **root interface** of the Java Collection Framework.

It defines a group of objects known as **elements**.

The Collection interface is implemented by various

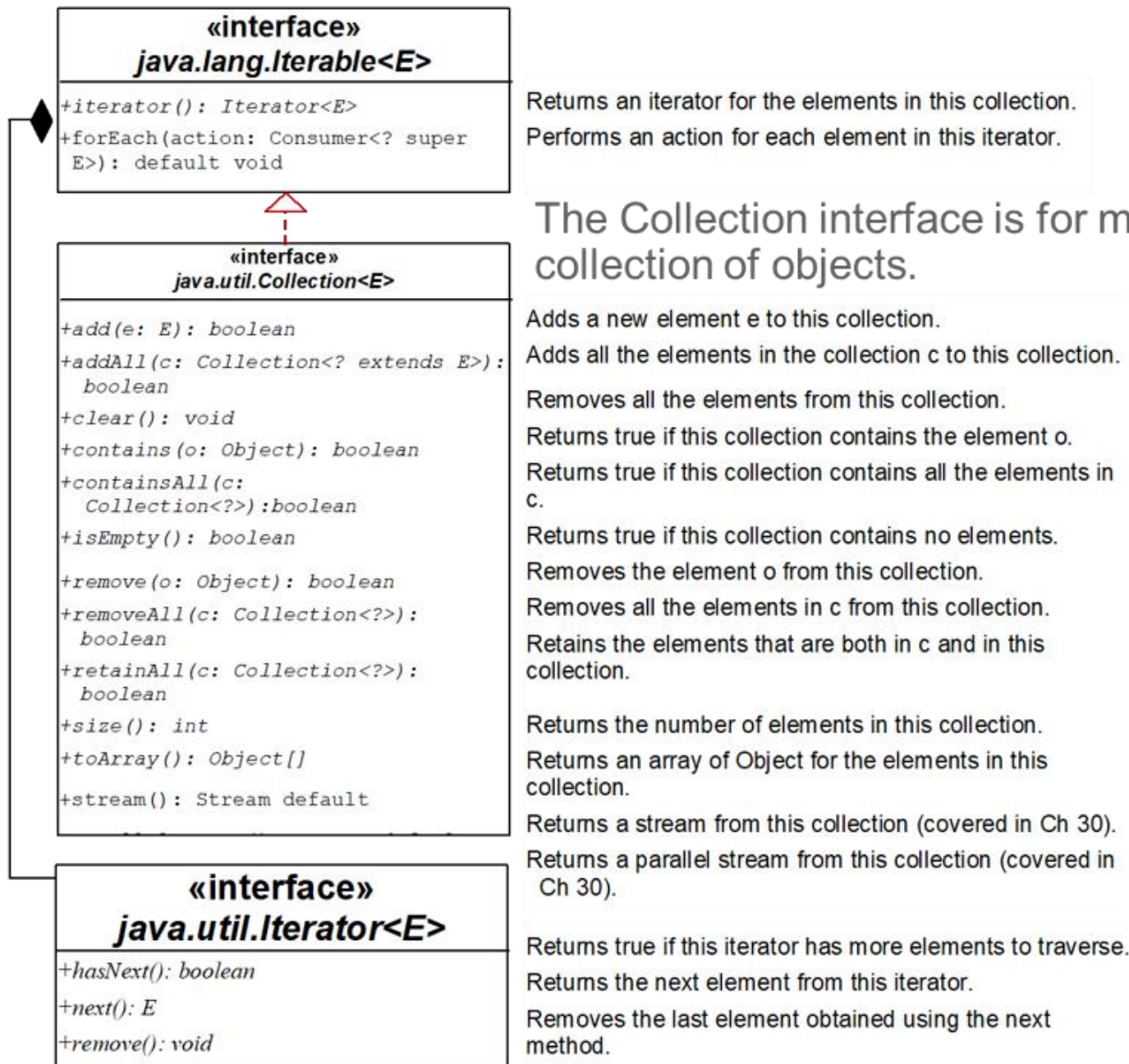
- sub-interfaces like List, Set, and Queue

<<Interface>>

Collection

[Base Interface]

```
add(E e)
addAll(Collection)
clear()
contains(Object o)
containsAll(Collection)
iterator()
remove(Object o)
removeAll(Collection)
removeIf(Predicate)
retainAll(Collection)
```



The Collection interface is for manipulating a collection of objects.

Overview of Classes and their characteristics

Interface	Class	Description
<u>List</u>	<u>ArrayList</u>	Resizable array that maintains order and allows duplicates
	<u>LinkedList</u>	List with fast insert and remove operations
<u>Set</u>	<u>HashSet</u>	Unordered collection of unique elements
	<u>TreeSet</u>	Sorted set of unique elements (natural order)
	<u>LinkedHashSet</u>	Maintains the order in which elements were inserted
<u>Map</u>	<u>HashMap</u>	Stores key/value pairs with no specific order
	<u>TreeMap</u>	Sorted map based on the natural order of keys
	<u>LinkedHashMap</u>	Maintains the order in which keys were inserted

Constructing a collection

Syntax:

- **Interface<Type> name = new Implementation<Type>();**

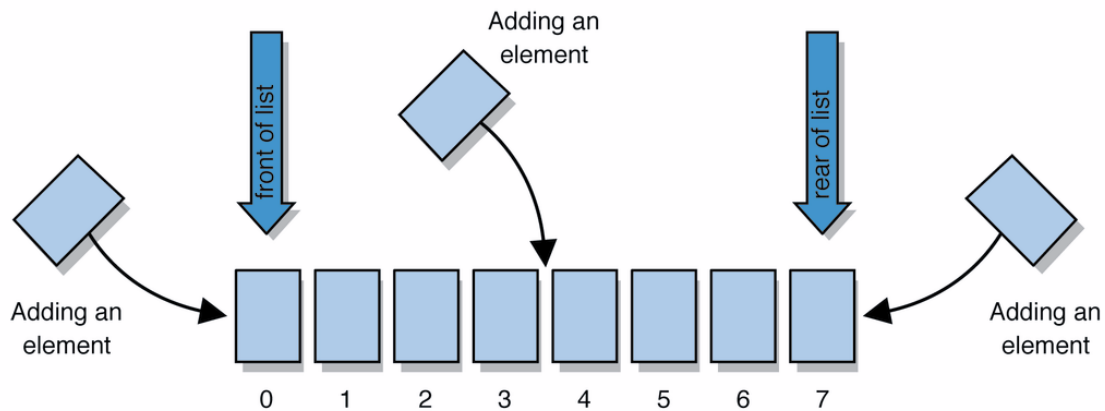
```
List<String> arrayList = new ArrayList<>();  
List<String> linkedList = new LinkedList<>();  
ArrayList<String> cars = new ArrayList<String>();
```

```
Set<String> hashSet = new HashSet<>();  
Set<String> treeSet = new TreeSet<>();
```

```
Map<String, Integer> hashMap = new HashMap<>();  
Map<String, Integer> treeMap = new TreeMap<>();
```

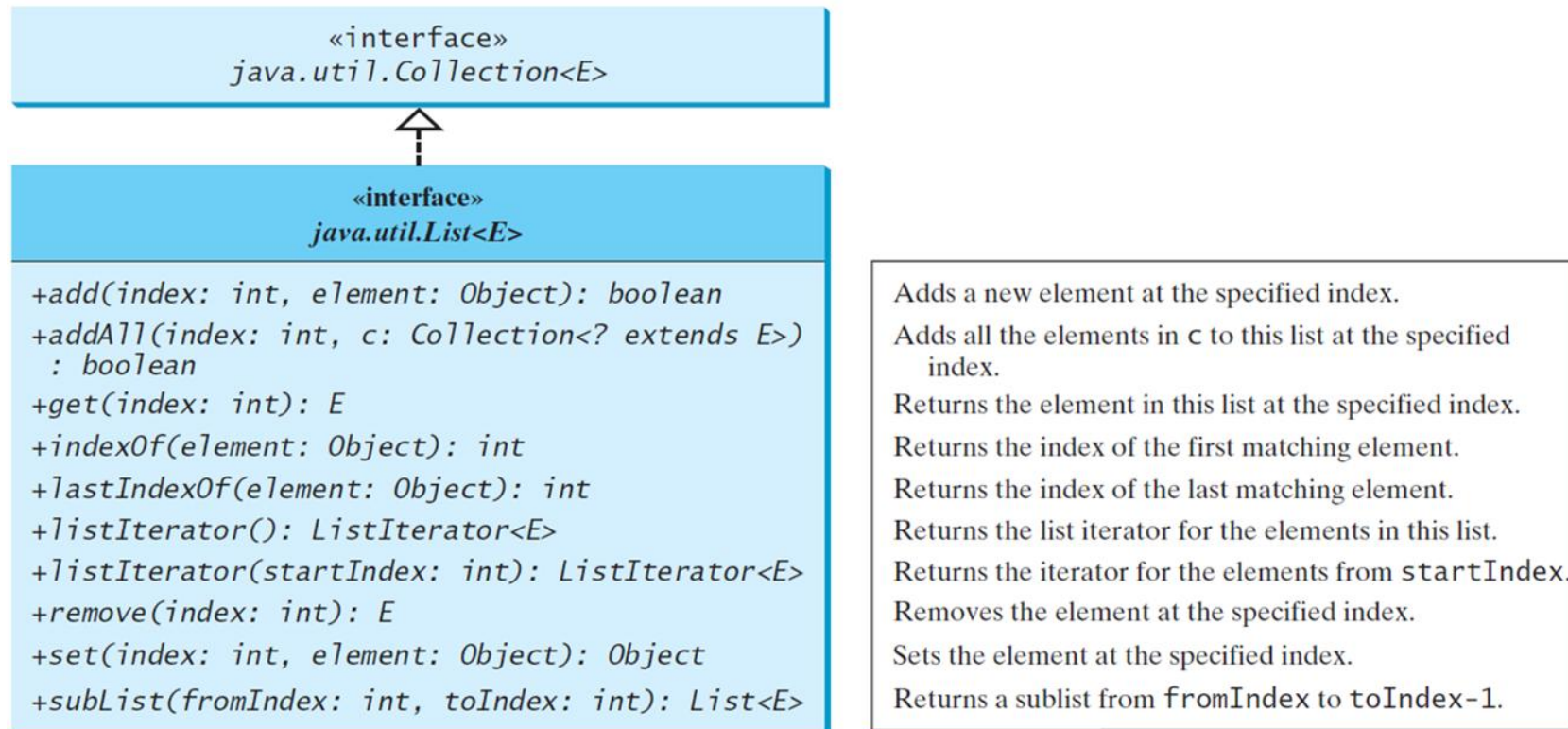
```
Queue<String> queue = new LinkedList<>();
```

Lists



- **List** is a collection storing an ordered sequence of elements
 - Each element is accessible by a 0-based **index**
 - A list has a **size** (number of elements that have been added)
 - Elements can be added to the front, back, or elsewhere
 - in Java, represented by the `List` interface, implemented by the **`ArrayList`** and **`LinkedList`** classes

The List Interface

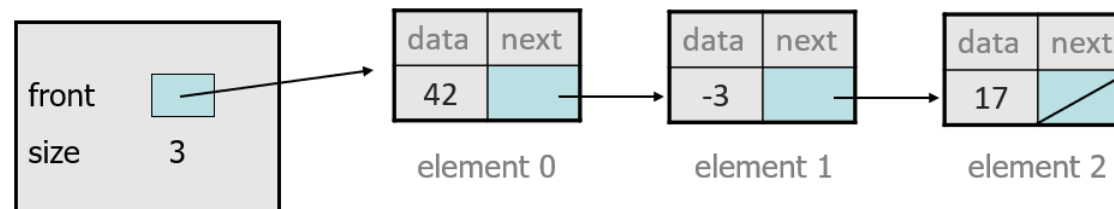


List implementations

- **ArrayList** is built using an internal "unfilled" array and a size field to remember how many elements have been added

<i>index</i>	0	1	2	3	4	5	6	7	8	9
<i>value</i>	42	-3	17	0	0	0	0	0	0	0
<i>size</i>	3									

- **LinkedList** is built using a chain of small "node" objects, one for each element of the data, with a link to a "next" node object



List Interface Example

```
import java.util.*;
public class CollectionsDemo {
    public static void main(String[] args) {
        List a1 = new ArrayList();
        a1.add("One");
        a1.add("Two");
        a1.add("Three");
        System.out.println("ArrayList Elements");
        System.out.print("\t" + a1);

        List l1 = new LinkedList();
        l1.add("Apple");
        l1.add("Banana");
        l1.add("Orange");
        System.out.println();
        System.out.println("LinkedList Elements");
        System.out.print("\t" + l1);
    }
}
```

Sets

Set is a collection of unique items.

The order may or may not be preserved.

HashSet:

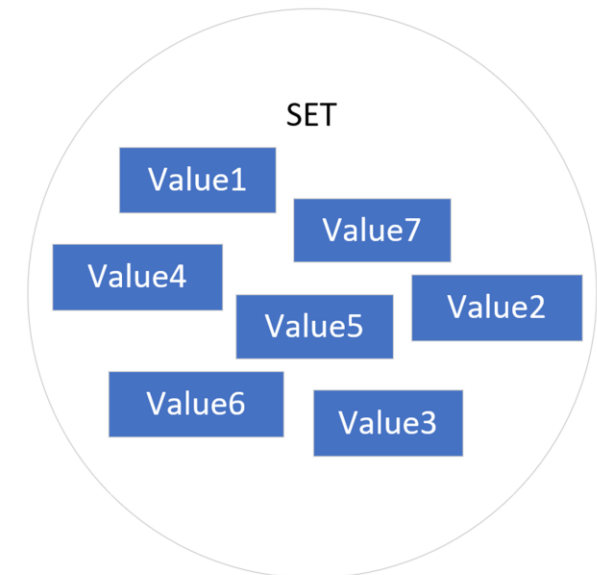
- Unordered
- No duplicates
- Fast access using hash

LinkedHashSet:

- Like HashSet, but maintains insertion order

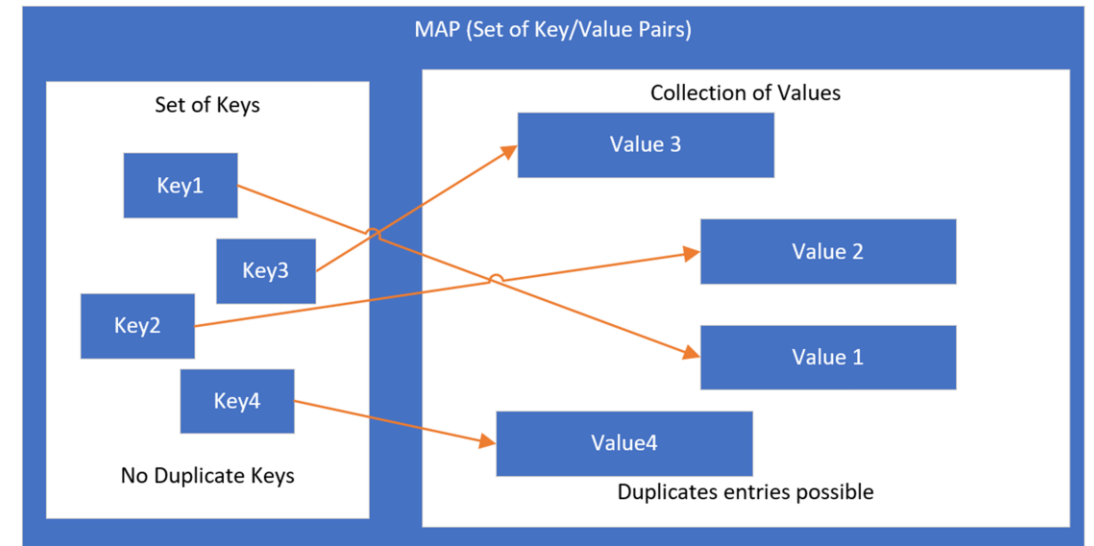
TreeSet:

- Maintains items in **sorted** order.



Maps

- A **map** is an object that stores associations between keys and values, or key/value pairs.
- Given a key, you can find its value. Both keys and values are objects.
- The keys must be unique, but the values may be duplicated. Some maps can accept a null key and null values, others cannot.
- **Maps don't implement the Iterable** interface. This means that you cannot cycle through a map using a for-each style for loop. Furthermore, you can't obtain an iterator to a map



Map

HashMap:

- Very fast for storing and retrieving **key-value pairs**.
- Doesn't maintain order.
- The HashMap class extends AbstractMap and implements the Map interface.
- It uses a hash table to store the map. This allows the execution time of get() and put() to remain constant even for large sets.
- HashMap is a generic class that has declaration:

```
class HashMap<K,V>
```

LinkedHashMap:

- Maintains insertion order

TreeMap:

- Keeps keys sorted.

Common Map Methods

Method	Example	Purpose
<code>put(K, V)</code>	<code>map.put("Alice", "A")</code>	Add/update entry
<code>get(K)</code>	<code>map.get("Alice")</code>	Get value by key
<code>remove(K)</code>	<code>map.remove("Bob")</code>	Remove entry
<code>containsKey(K)</code>	<code>map.containsKey("Alice")</code>	Check if key exists
<code>containsValue(V)</code>	<code>map.containsValue("A")</code>	Check if value exists
<code>replace(K, V)</code>	<code>map.replace("Alice", "B")</code>	Replace value
<code>putIfAbsent(K, V)</code>	<code>map.putIfAbsent("Eve", "C")</code>	Add only if key not present
<code>keySet()</code>	<code>map.keySet()</code>	Get all keys
<code>values()</code>	<code>map.values()</code>	Get all values
<code>entrySet()</code>	<code>map.entrySet()</code>	Get all key-value pairs
<code>clear()</code>	<code>map.clear()</code>	Remove all entries
<code>isEmpty()</code>	<code>map.isEmpty()</code>	Check if empty
<code>size()</code>	<code>map.size()</code>	Count entries

```

import java.util.*;
class HashMapDemo {
    public static void main(String args[]) {
        // Create a hash map.
        HashMap<String, Double> hm = new HashMap<String, Double>();
        // Put elements to the map
        hm.put("John Doe", new Double(3434.34));
        hm.put("Tom Smith", new Double(123.22));
        hm.put("Jane Baker", new Double(1378.00));
        hm.put("Tod Hall", new Double(99.22));
        hm.put("Ralph Smith", new Double(-19.08));
        // Get a set of the entries.
        Set<Map.Entry<String, Double>> set = hm.entrySet();
        // Display the set.
        for(Map.Entry<String, Double> me : set) {
            System.out.print(me.getKey() + ": ");
            System.out.println(me.getValue());
        }
        System.out.println();
        //Deposit 1000 into John Doe's account.
        double balance = hm.get("John Doe");
        hm.put("John Doe", balance + 1000);
        System.out.println("John Doe's new balance: " +
            hm.get("John Doe"));
    }
}

```

Queue (1/2)

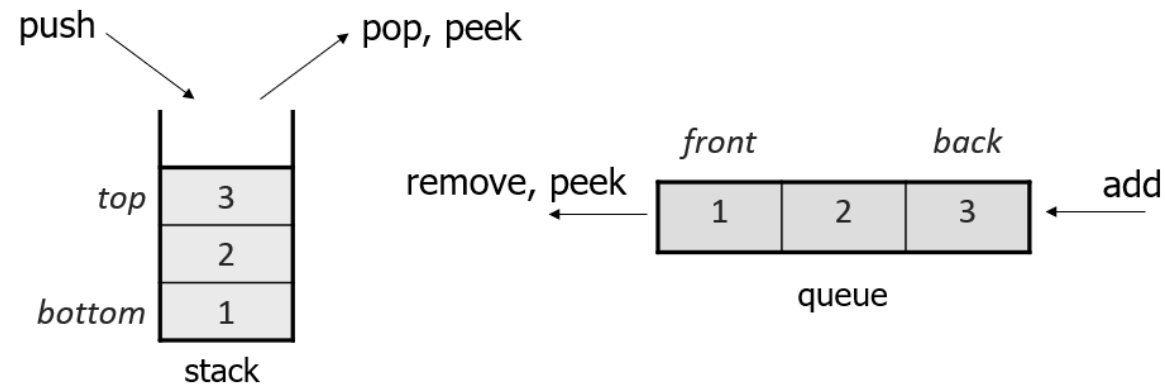
Stores elements to be processed, usually in the order they came in (First-In-First-Out).

Stack: Retrieves elements in the reverse of the order they were added.

Queue: Retrieves elements in the same order they were added.

PriorityQueue: Elements are ordered by priority, not insertion.

ArrayDeque: Double-ended queue, efficient insertion/removal from both ends.



Queue (2/2)

Queue interface extends **Collection** and declares the behaviour of a queue, which is often a first-in, first-out list.

LinkedList and **PriorityQueue** are the two classes which implements Queue interface

Queue is declared as follows:

```
Queue<E> q = new LinkedList<E>();
```

```
Queue<E> q = new PriorityQueue<E>();
```

where E specifies the type of object.

Queue Implementation Comparison

Feature / Methods	LinkedList (Queue)	PriorityQueue	ArrayDeque
Ordering	Maintains insertion order (FIFO)	Orders elements by natural order or a Comparator Underlying DS: Heap	Maintains order of insertion, but allows adding/removing from both ends
Null elements	✓ Allowed (but discouraged)	✗ Not allowed	✗ Not allowed
Duplicates	✓ Allowed	✓ Allowed	✓ Allowed
Performance	Fast insertion/removal at ends	Efficient retrieval of smallest/largest	Very fast for both ends (better than LinkedList)
Use cases	Bank queue, customer service line	Hospital ER, job scheduler	Browser history, task scheduling, stack + queue in one

Queue methods

E element()

Returns the element at the head of the queue. The element is not removed. It throws NoSuchElementException if the queue is empty.

boolean offer(E obj)

Attempts to add obj to the queue. Returns true if obj was added and false otherwise.

E peek()

Returns the element at the head of the queue. It returns null if the queue is empty. The element is not removed.

E poll()

Returns the element at the head of the queue, removing the element in the process. It returns null if the queue is empty.

E remove()

Returns the element at the head of the queue, returning the element in the process. It throws NoSuchElementException if the queue is empty.

Summary of main methods in Queue

Category	Throws Exception on Failure	Returns Special Value on Failure
Adding	<code>add(e)</code>	<code>offer(e)</code>
Removing	<code>remove()</code>	<code>poll()</code>
Inspecting	<code>element()</code>	<code>peek()</code>

Method	Description
<code>offerFirst(E e)</code>	Inserts at front
<code>offerLast(E e)</code>	Inserts at end
<code>peekFirst()</code>	Returns first element
<code>peekLast()</code>	Returns last element
<code>pollFirst()</code>	Removes first element
<code>pollLast()</code>	Removes last element

Queue Example

```
import java.util.*;
public class QueueDemo {
    public static void main(String[] args) {
        Queue<String> q = new LinkedList<String>();
        q.add("Tom");
        q.add("Jerry");
        q.add("Mike");
        q.add("Steve");
        q.add("Harry");
        System.out.println("Elements in Queue:"+q);
        System.out.println("Removed element: "+q.remove());
        System.out.println("Head: "+q.element());
        System.out.println("poll(): "+q.poll());
        System.out.println("peek(): "+q.peek());
        System.out.println("Elements in Queue:"+q);
    }
}
```

```
Elements in Queue:[Tom, Jerry, Mike, Steve, Harry]
Removed element: Tom
Head: Jerry
poll(): Jerry
peek(): Mike
Elements in Queue:[Mike, Steve, Harry]
```

Which Collection to choose?

List: An ordered collection that allows duplicates.

- **ArrayList:** Best for fast access by index.
- **LinkedList:** Best for frequent additions/removals from the middle.

Set: A collection of unique items where duplicates are not allowed.

- **HashSet:** Very fast, but makes no guarantee about order.
- **TreeSet:** Keeps the elements in a sorted order.

Map: A collection of key-value pairs where keys must be unique.

- **HashMap:** Very fast, but makes no guarantee about key order.
- **TreeMap:** Keeps the entries sorted by their keys.

Algorithms Provided by Collections

The Collections class provides **static methods** for algorithms:

```
import java.util.*;
public class CollectionsDemo {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>(Arrays.asList(5, 2, 9, 1));
        Collections.sort(numbers);    // Sort ascending
        System.out.println("Sorted: " + numbers);
        Collections.reverse(numbers); // Reverse
        System.out.println("Reversed: " + numbers);
        int max = Collections.max(numbers);
        System.out.println("Max: " + max);
        int min = Collections.min(numbers);
        System.out.println("Min: " + min);
    }
}
```

Iterating Elements

For-each loop:

- Simple, read-only

Iterator:

- Forward-only, safe remove

ListIterator:

- Forward/backward traversal
- modify elements

Iterator interface

An **Iterator** is an **interface in Java** that provides a way to **traverse elements of a Collection (like List, Set, or Queue) sequentially** without exposing the underlying structure.

Each of the collection classes provides an **iterator()** method that returns an iterator to the start of the collection. By using this iterator object, you can access each element in the collection, one element at a time.

Interface Signature:

```
public interface Iterator<E> {  
    boolean hasNext();  
    E next();  
    void remove(); // optional operation  
}
```

Need of an Iterator

Unified traversal: Works for any collection (List, Set, Queue).

Avoids errors: Safe way to remove elements while iterating (better than for loop).

No index required: Especially useful for collections like Set that don't support indexing.

Methods of Iterator interface

To use an iterator to cycle through the contents of a collection, follow these steps:

1. Obtain an iterator to the start of the collection by calling the collection's `iterator( )` method.
2. Set up a loop that makes a call to `hasNext( )`. Have the loop iterate as long as `hasNext( )` returns true.
3. Within the loop, obtain each element by calling `next( )`.

Method	Description
hasNext()	Returns true if there are more elements to iterate over.
next()	Returns the next element in the iteration. Throws NoSuchElementException if no elements are left.
remove()	Removes the last element returned by next(). Can only be called once per call to next(). Throws IllegalStateException if called improperly.

Iterator Example

```
import java.util.*;
public class IteratorDemo {
    public static void main(String args[]) {
        ArrayList<String> al = new ArrayList<String>();
        al.add("C");
        al.add("A");
        al.add("E");
        al.add("B");
        al.add("D");
        al.add("F");
        System.out.print("Contents of List: ");
        Iterator<String> itr = al.iterator();
        while(itr.hasNext()) {
            Object element = itr.next();
            System.out.print(element + " ");
        }
    }
}
```

ListIterator

ListIterator is an interface in Java that **extends Iterator**.

It is designed specifically for Lists (e.g., ArrayList, LinkedList).

Unlike Iterator, ListIterator allows bidirectional traversal and modification of elements during iteration.

Interface Signature:

```
public interface ListIterator<E> extends Iterator<E> {  
    boolean hasPrevious();  
    E previous();  
    int nextIndex();  
    int previousIndex();  
    void set(E e);  
    void add(E e);  
}
```

Key features of ListIterator

Feature	Description
Bidirectional traversal	Can move forward (<code>next()</code>) or backward (<code>previous()</code>) through the list.
Index information	<code>nextIndex()</code> and <code>previousIndex()</code> give the position of elements.
Modify during iteration	<code>set()</code> replaces the last element returned by <code>next()</code> or <code>previous()</code> .
Add elements	<code>add()</code> inserts a new element at the current position.
Works only with List	Unlike Iterator, it cannot be used with Set or Queue.

Key differences between Iterator and ListIterator

Feature	Iterator	ListIterator
Traversal direction	Forward only	Forward and backward
Modify elements	<code>remove()</code> only	<code>remove()</code> , <code>set()</code> , <code>add()</code>
Applicable collections	Any Collection	List only
Index info	No	<code>nextIndex()</code> , <code>previousIndex()</code>